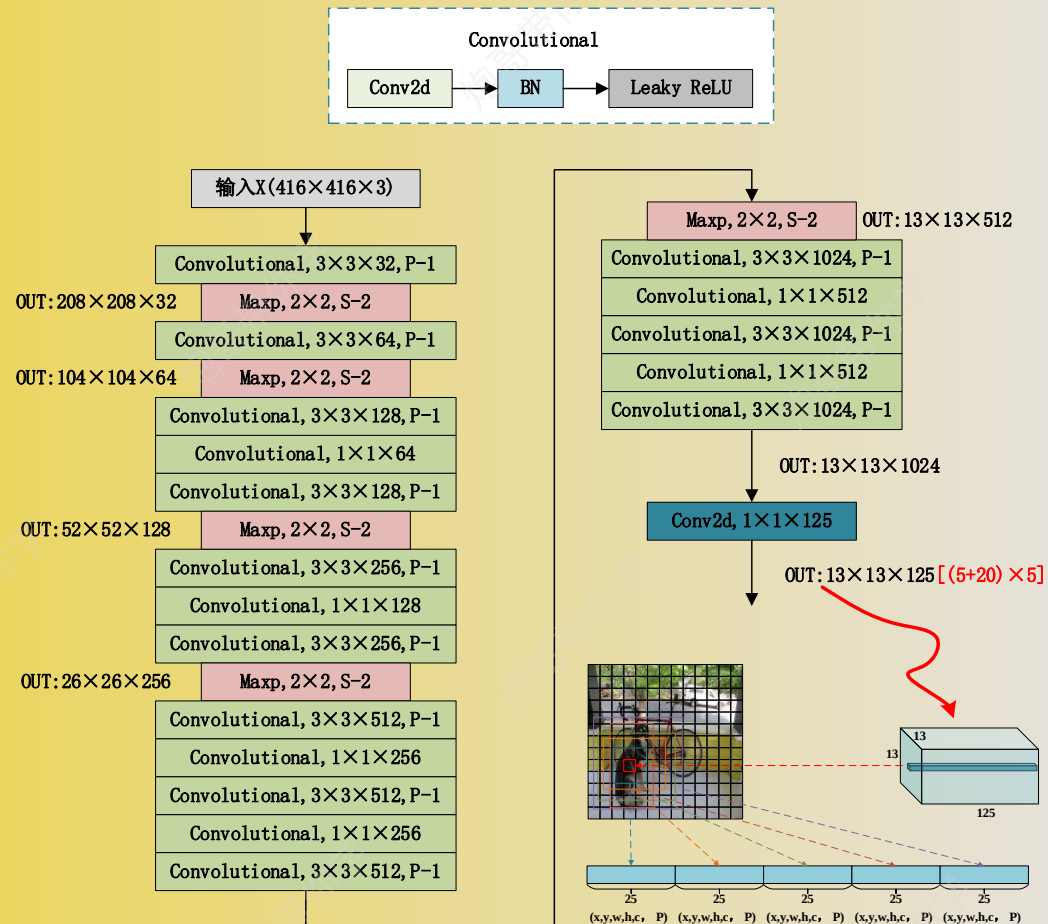
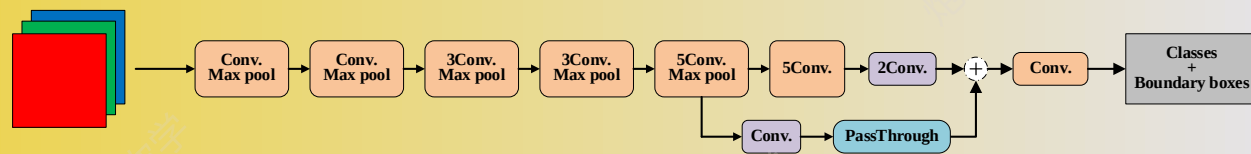




非完整版的DarkNet-19



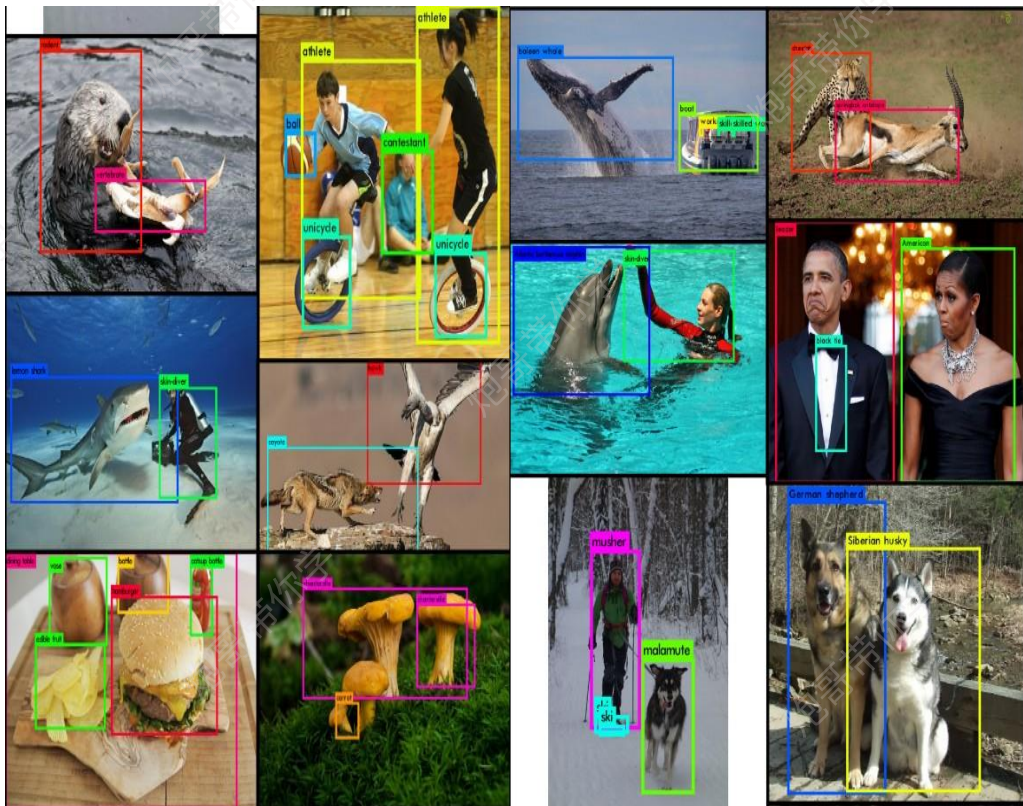
第3章

Yolov2

目标检测

算法原理

Yolov2目标检测的背景



YOLOv2 是YOLO的第二个版本，其目标是显著提高准确性，同时使其更快。

YOLOv2 在 YOLOv1的基础上做了许多改进，其中在 VOC2007 资料集上的mAP 由 63.4% 提升到 78.6%，并且保持检测速度。从预测更准确（Better），速度更快（Faster），识别物体更多（Stronger）这三个方面进行了改进。其中识别更多目标物体也就是扩展到能够检测9000种不同物体，称之为YOLO9000。

Yolov2模型改进点

	YOLO									YOLOv2
batch norm?		✓	✓	✓	✓	✓	✓	✓	✓	✓
hi-res classifier?			✓	✓	✓	✓	✓	✓	✓	✓
convolutional?				✓	✓	✓	✓	✓	✓	✓
anchor boxes?				✓	✓					
new network?					✓	✓	✓	✓	✓	✓
dimension priors?						✓	✓	✓	✓	✓
location prediction?						✓	✓	✓	✓	✓
passthrough?							✓	✓	✓	✓
multi-scale?								✓	✓	✓
hi-res detector?									✓	✓
VOC2007 mAP	63.4	65.8	69.5	69.2	69.6	74.4	75.4	76.8		78.6

batch norm: Yolov2相较于Yolov1添加了BN层;

Hi-res classifier: Yolov2相较于Yolov1采用更高分辨率的网络进行分类主干网络的训练;

Convolutional+anchor: Yolov2相较于Yolov1去除全连接层, 采用卷积层进行模型的输出;
同时采用与锚框(预选框)进行bounding box的预测;

new network: 采用新的网络架构Darknet-19

dimension priors: 采用k-means聚类方法对训练集中的标准框做了聚类分析, 获取anchor boxes;

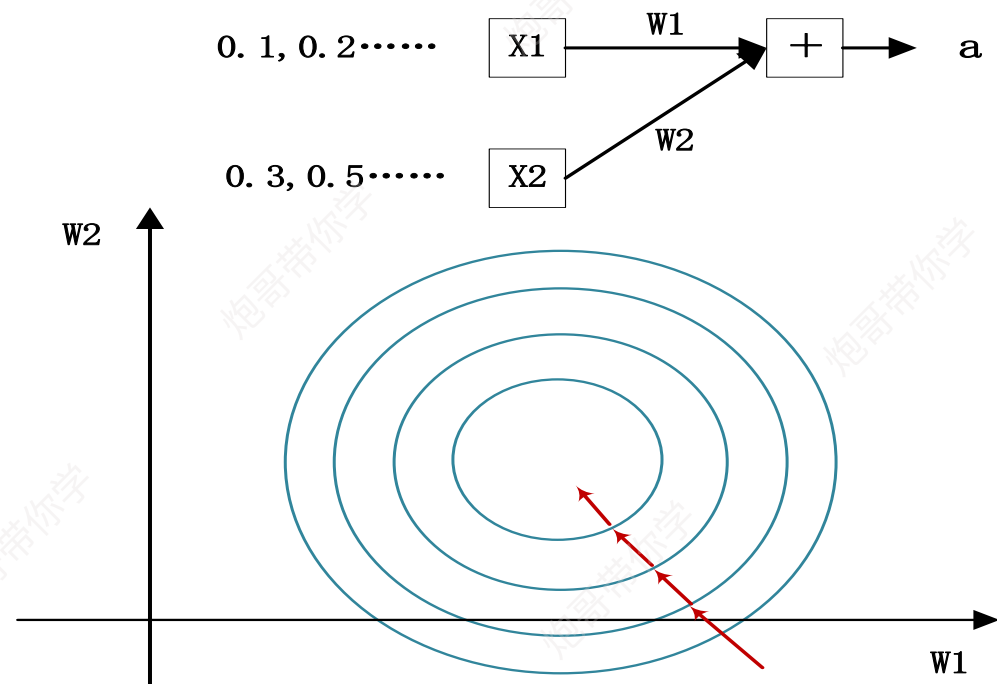
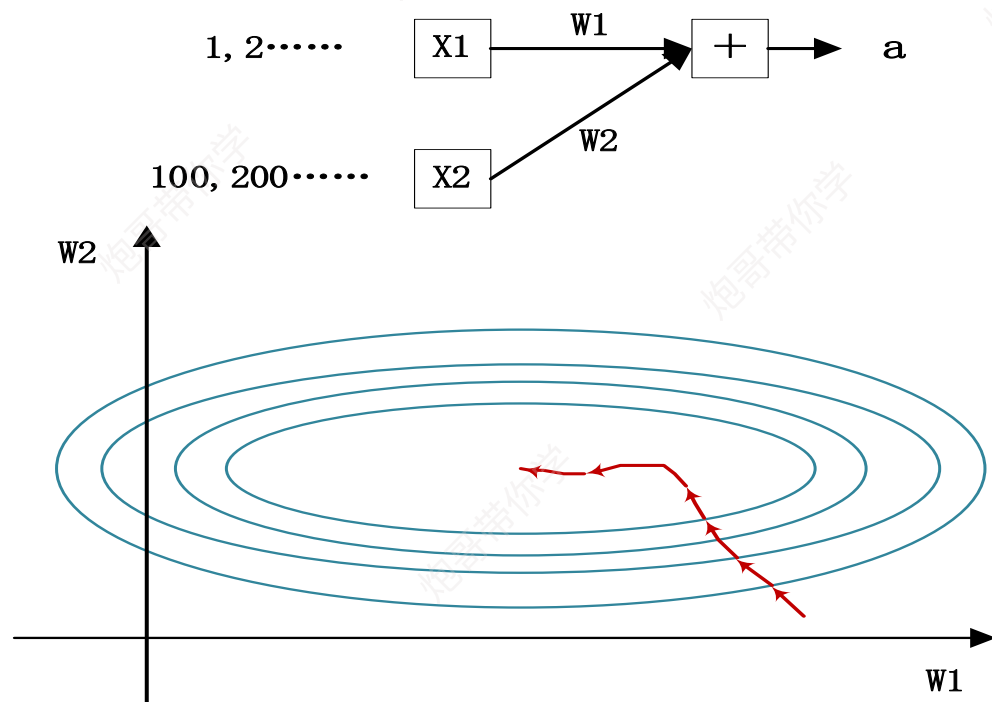
location prediction: 使用sigmoid函数处理位置预测值。

passthrough: passthrough网络模型的连接方式(类似resnet);

multi-scale: 多尺度输入数据训练模型;

hi-res detector: Yolov2相较于Yolov1采用更高分辨率的网络进行检测主干网络的训练

Yolov2改进点- Batch norm



我们知道网络一旦train起来，那么参数就要发生更新，除了输入层的数据外(因为输入层数据，我们已经人为的为每个样本归一化)，后面网络每一层的输入数据分布是一直在发生变化的，因为在训练的时候，前面层训练参数的更新将导致后面层输入数据分布的变化。

以网络第二层为例：网络的第二层输入，是由第一层的参数和input计算得到的，而第一层的参数在整个训练过程中一直在变化，因此必然会引起后面每一层输入数据分布的改变。BN的提出，就是要解决在训练过程中，中间层数据分布发生改变的情况。

Yolov2改进点- Batch norm, BN怎么做?

如上图所示, BN步骤主要分为4步:

- (1) 求每一个训练批次数据的均值
- (2) 求每一个训练批次数据的方差
- (3) 使用求得的均值和方差对该批次的训练数据做归一化, 获得0-1分布。其中 ϵ 是为了避免除数为0时所使用的微小正数。
- (4) 尺度变换和偏移: 将 x_i 乘以 γ 调整数值大小, 再加上 β 增加偏移后得到 y_i , 这里的 γ 是尺度因子, β 是平移因子。这一步是BN的精髓, 由于归一化后的 x_i 基本会被限制在正态分布下, 使得网络的表达能力下降。为解决该问题, 我们引入两个新的参数: γ, β 。 γ 和 β 是在训练时网络自己学习得到的。

输入: 批处理 (mini-batch) 输入 $x: \mathcal{B} = \{x_1, \dots, x_m\}$

输出: 规范化后的网络响应 $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

1: $\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$ // 计算批处理数据均值

2: $\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$ // 计算批处理数据方差

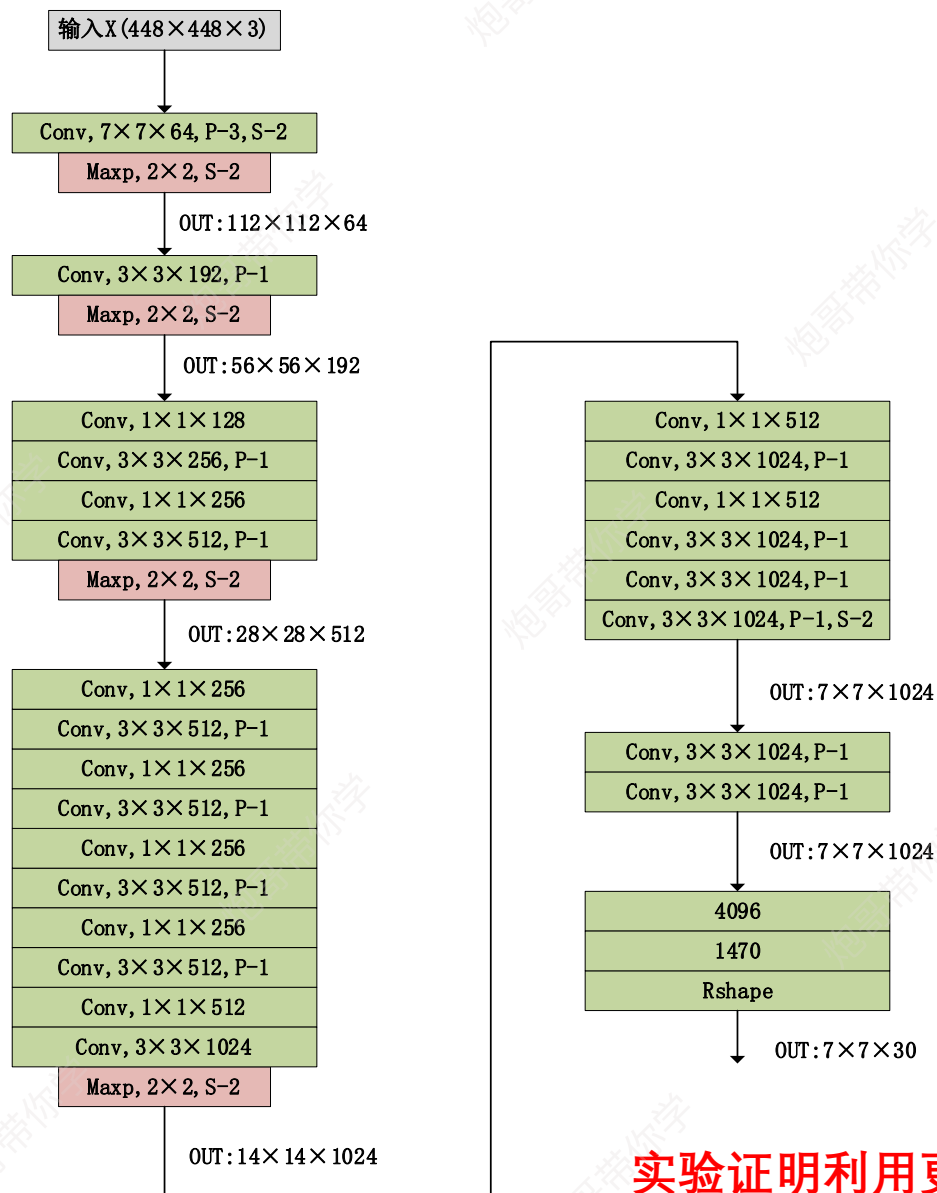
3: $\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$ // 规范化

4: $y_i \leftarrow \gamma \hat{x}_i + \beta = \text{BN}_{\gamma, \beta}(x_i)$ // 尺度变换和偏移

5: **return** 学习的参数 γ 和 β .

实验证明添加了BN层的同时去除Drouput可以提高2%的mAP。

Yolov2改进点- Hi-res classifier



训练数据： ImageNet，该数据为1000分类，利用该数据对模型进行分类模型的训练，使得模型有更好的初始化权重。

Yolov1:

分类模型：

- 1、训练输入数据大小， 224×224 。
- 2、训练轮次大约一周的时间， top-5准确率88%。

检测模型训练：

训练周期约为135轮，数据大小为448。

Yolov2:

分类模型：

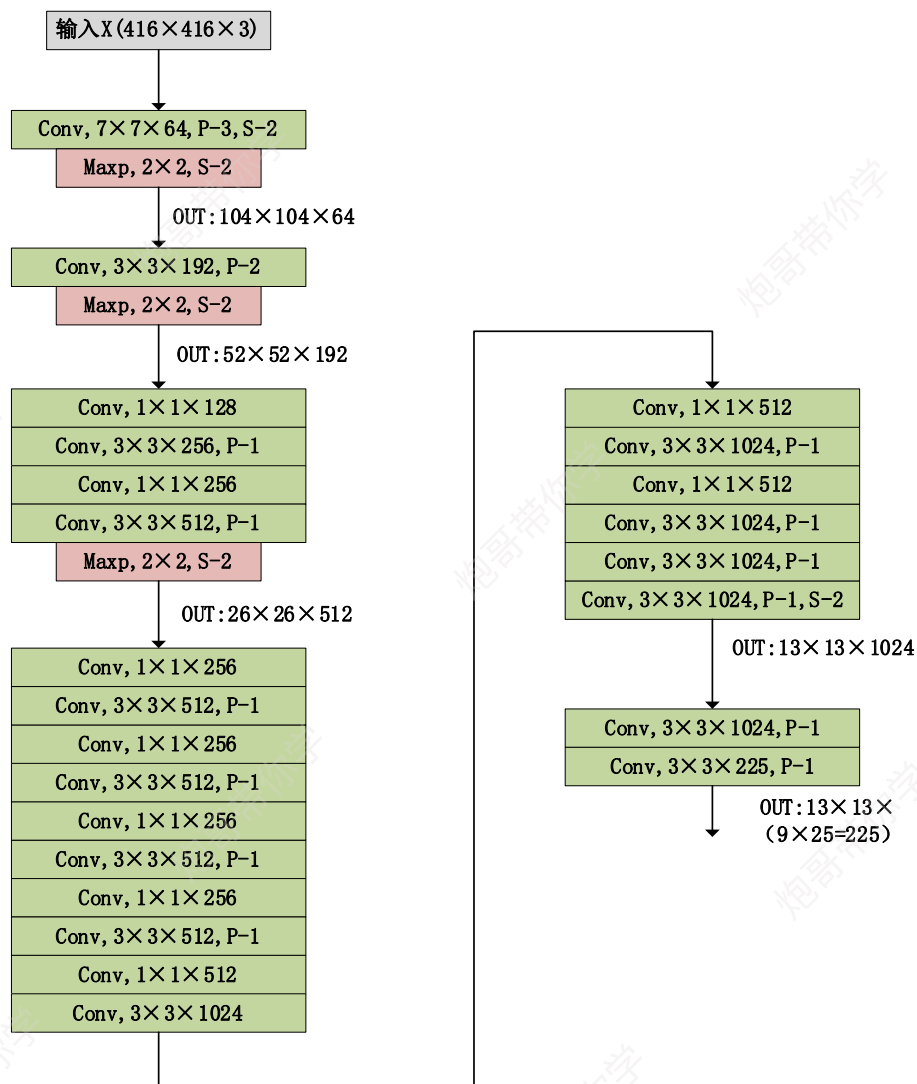
- 1、训练输入数据大小， 224×224 ，训练160轮次。
- 2、训练输入数据大小， 448×448 ，训练10轮次。
- 3、高分辨率下训练的分类网络在top-1准确率76.5%， top-5准确率93.3%。

检测模型训练：

利用多尺度训练的方法一共训练160轮。

实验证明利用更高分辨率的图像mAP提升了约4%

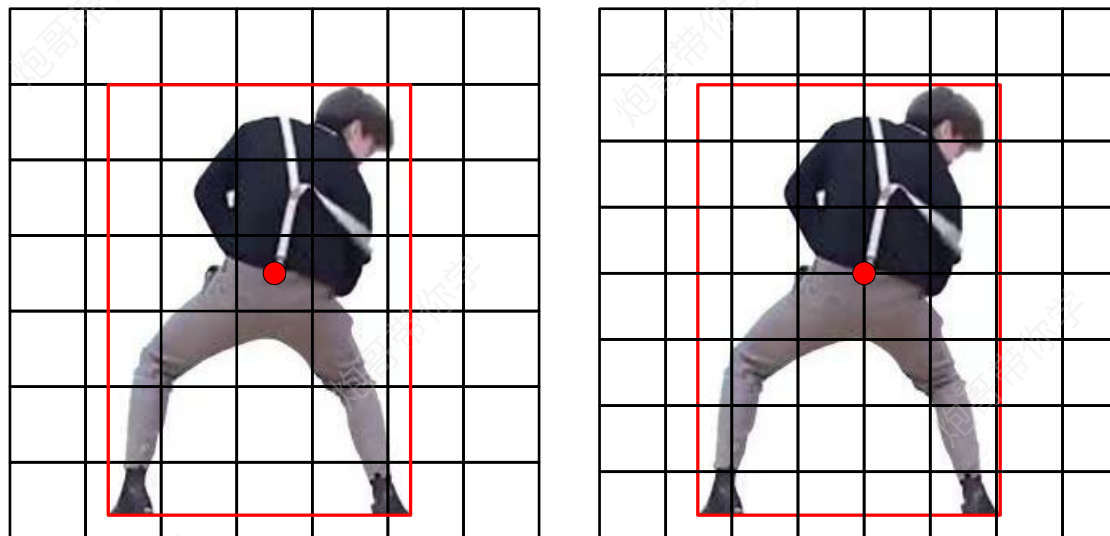
Yolov2改进点-Convolutional+Anchor boxes



YOLOv1 最终是通过全连接网络直接预测目标位置。

YOLO 作者把 YOLOv1 进行了改造:

- 1、最后的全连接层去掉。
- 2、移除最后一个 pool 层, 使得卷积层输出更高分辨率。
- 3、用 416×416 大小的输入代替原来 448×448 。
- 4、将分类和空间检测解耦, 利用 anchor box 来预测目标的空间位置和类别。



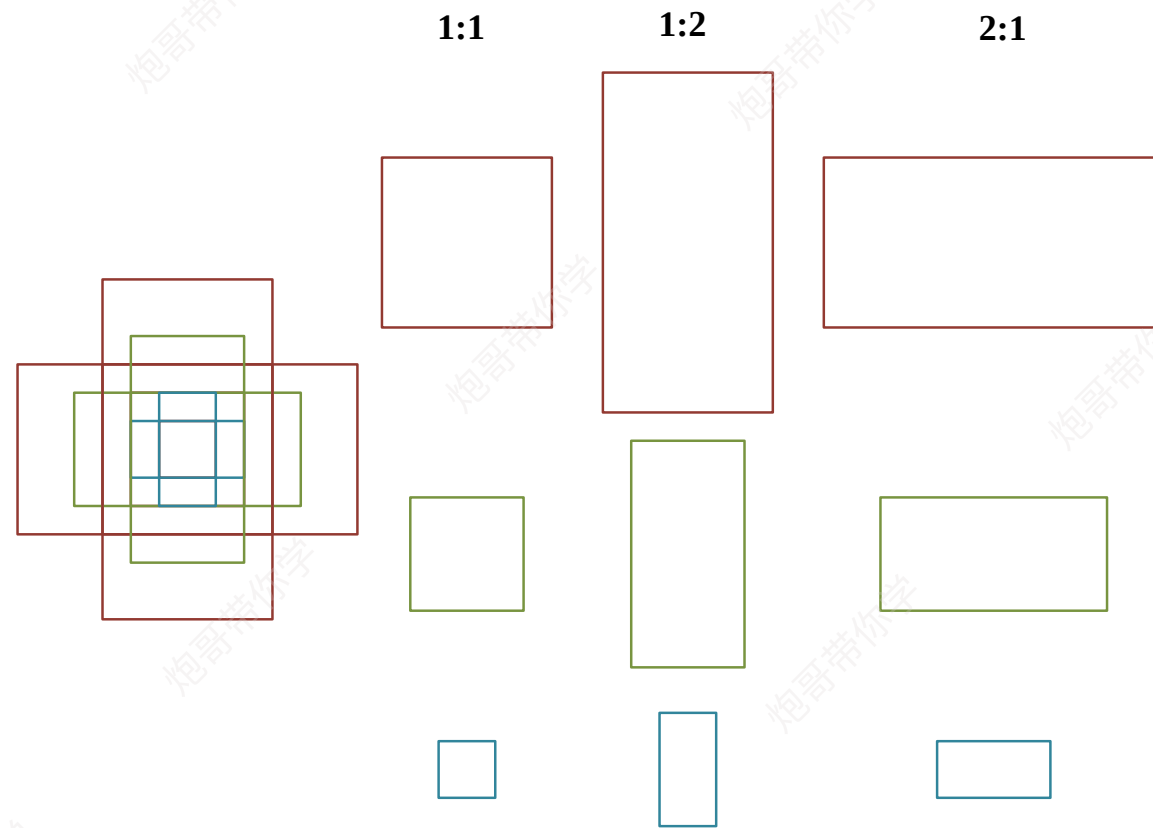
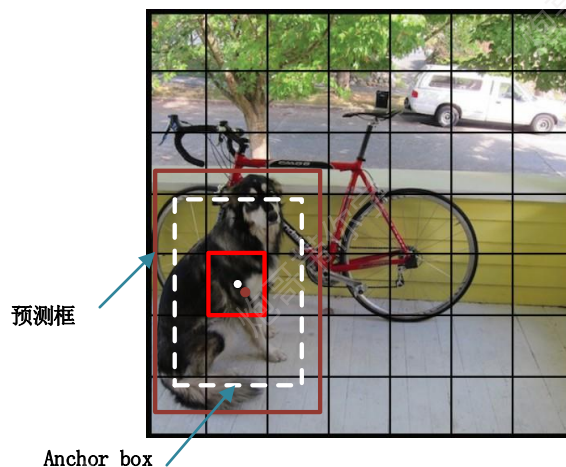
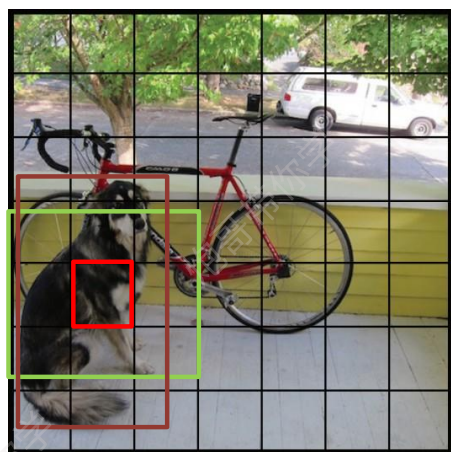
Yolov2改进点-Anchor boxes介绍

Yolov2借鉴了Faster R-CNN和SSD的做法利用anchor框进行预测。

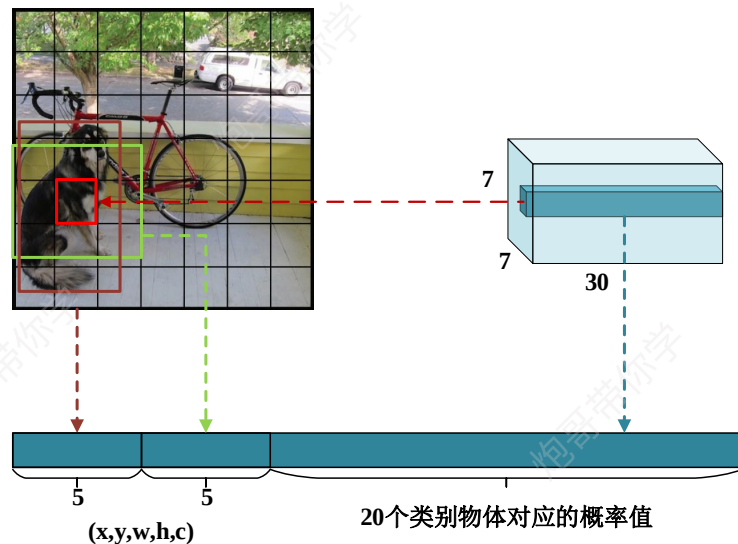
在Yolov1中是直接预测出物体的检测框大小和坐标，由于物体的大小不一，直接根据主干网络提取的信息进行矩形框和坐标的预测是一件较难的事情。

如果现在有一个合理的box，让你基于这个box来预测真实检测框，相比较直接预测检测框要简单一些。

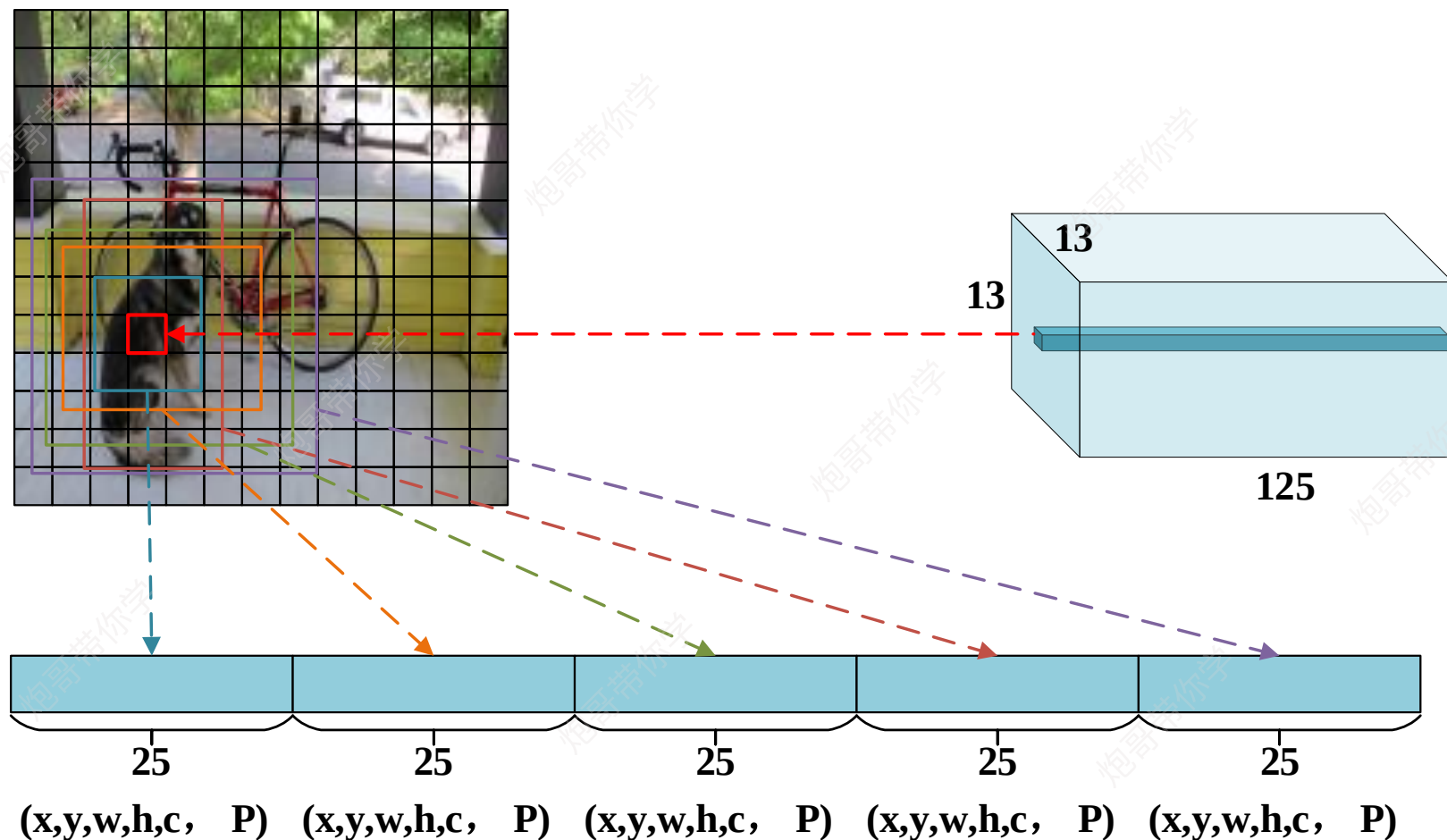
三种尺寸分别是小（蓝128）中（绿256）大（红512），三个比例分别是1:1， 1:2， 2:1。3×3的组合总共有9种anchor。



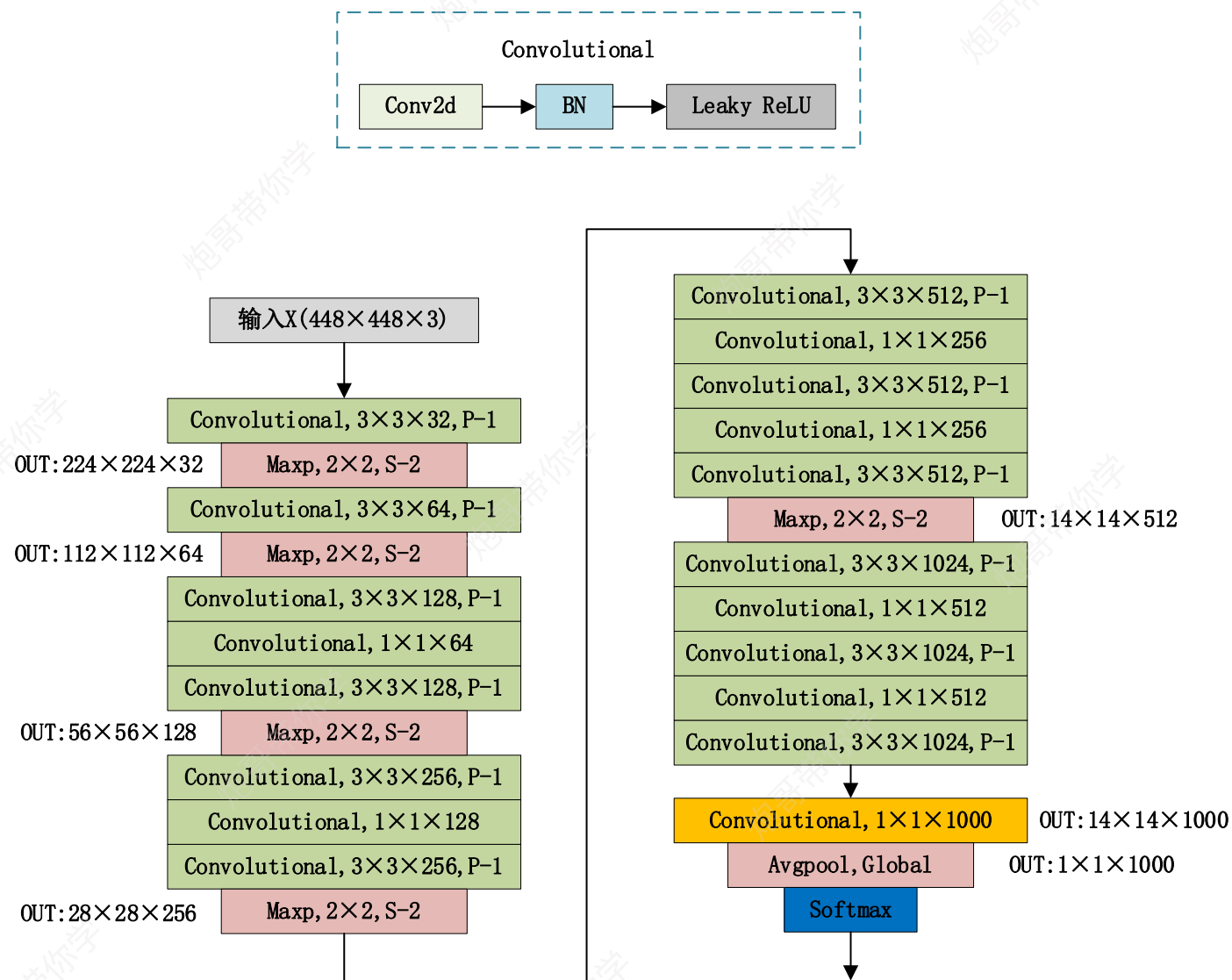
Yolov2改进点-如何利用Anchor boxes做预测?



Convolutional+Anchor使得MAP值下降了一点（69.5降到69.2），但是提高了recall（81%提高到88%）。这使得模型有更大的改进空间



Yolov2改进点-- Darknet-19-分类模型



DarkNet-19分类模型

Darknet-19分类模型中由于没有全连接层，因此对于模型的输入是图片大小是没有特殊的规定要求的，不管输入的图片是在448×448还是224×224，最终经最后一层1×1卷积层和全局平均池化层都会被转化为1×1×1000的向量，从而进行分类工作。

Yolov2改进点- dimension priors

在引入 Anchor boxes 到 YOLO 的过程，遇到了 2 个问题：

第1个问题就是 Anchor BOX 的尺寸是手选的，如果一开始的时候就分配好合理的先验尺寸，那么这无疑会加快学习的速度。相比于人为指定 anchor box 的尺寸比例，YOLO 作者想到了一个自动化的手段，那就是选择 k-means 聚类手段，在数据训练集中运行 k-means 算法，可以得到 k 个尺寸比例。

第2个问题就是模型不稳定，尤其是在训练刚开始的时候。作者认为这种不稳定主要来自预测box的(x,y)值。

解决第1个问题：

K-means算法是典型的基于距离（欧式距离、曼哈顿距离）的聚类算法，采用距离作为相似性的评价指标，即认为两个对象的距离越近，其相似度就越大。

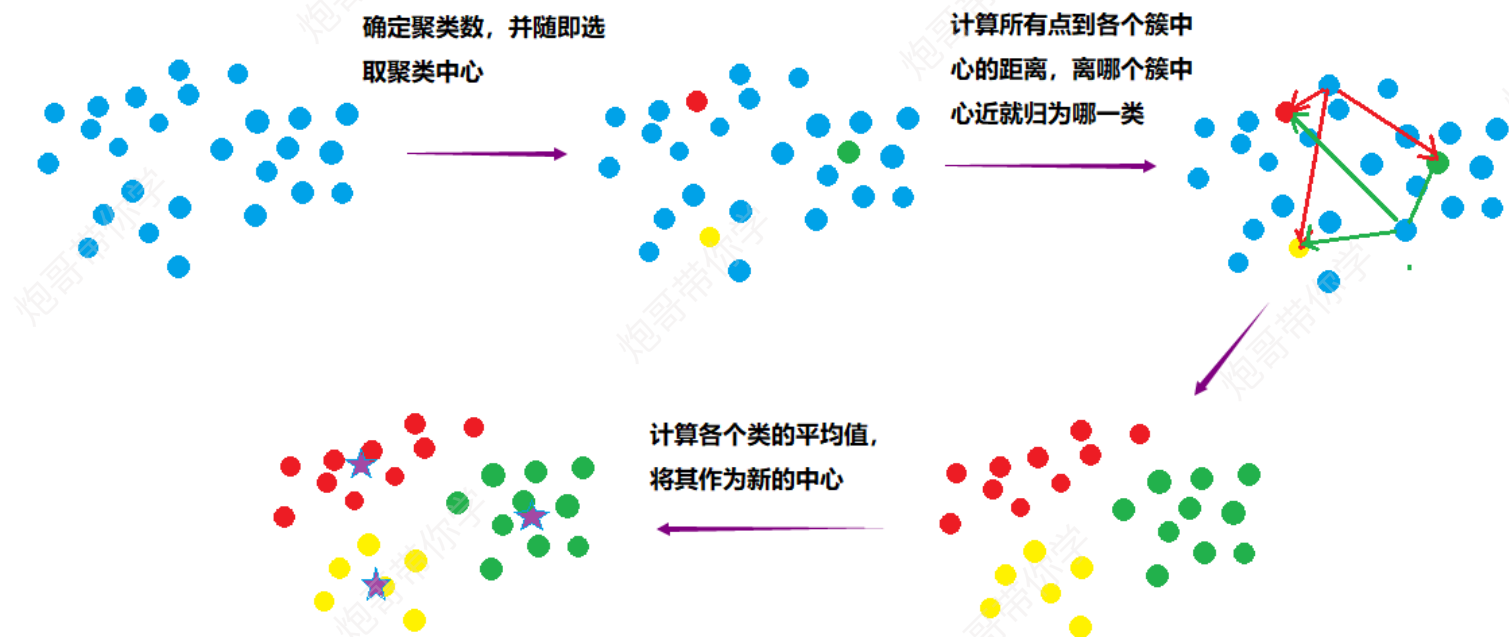
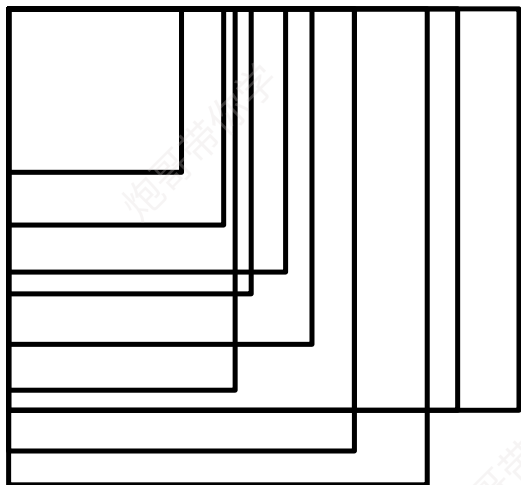
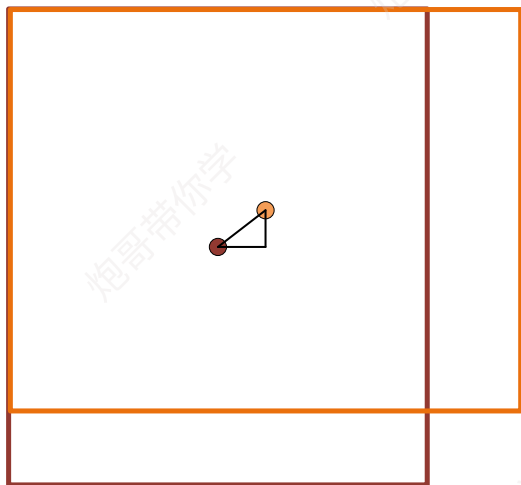
K-mean算法步骤如下：

- 1、先定义总共有多少个簇类，随机选取K个簇中心。
- 2、分别计算所有样本到随机选取的K个簇中心的距离。
- 3、样本离哪个中心近就被分到哪个簇中心。
- 4、计算各个中心样本的均值（最简单的方法就是求样本每个点的平均值）作为新的簇心。
- 5、重复2、3、4直到新的中心和原来的中心基本不变化的时候，算法结束。

算法结束条件：

- 1、当每个簇的质心，不再改变时就可以停止k-menas。
- 2、当循环次数达到事先规定的次数时，停止k-means

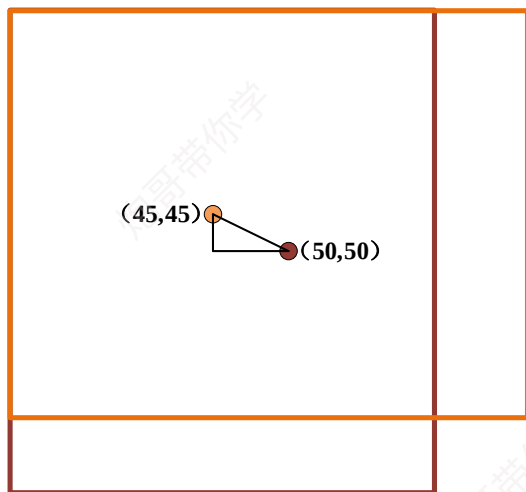
Yolov2改进点- 如何利用k-means进行真实框聚类



利用真实框进行K-mean算法步骤如下：

- 1、先定义总共有多少个簇类，随机选取K个样本为簇中心。
- 2、分别计算所有样本到随机选取的K个簇中心的欧式距离。
- 3、样本离哪个中心近就被分到哪个簇中心。
- 4、计算各个中心样本的均值作为新的簇心。
- 5、重复2、3、4直到新的中心和原来的中心基本不变化的时候，算法结束。

Yolov2改进点- 聚类利用欧式距离问题及其改进



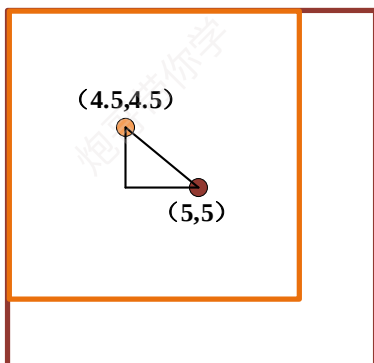
距离度量如果使用标准的欧氏距离，大box会比小box产生更多的误差。例50x50和45x45的box与5x5和4.5x4.5的box。因此这里使用其他的距离度量公式。

例如：

$$\sqrt{(50-45)^2 + (50-45)^2} = \sqrt{50}$$

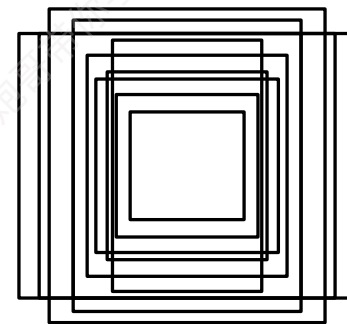
$$\sqrt{(5-4.5)^2 + (5-4.5)^2} = \sqrt{0.5}$$

很显然，大的box比小的box产生更多的误差，这会导致，假设k等于5的情况下，大的box被分成4个簇，而中小的box被分到一个簇中。



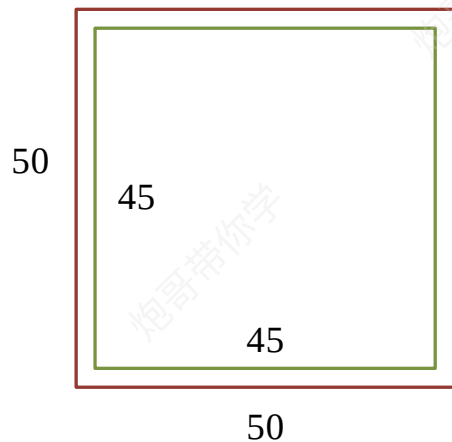
为了寻找合理的几类anchor box，应该抓住不同anchor box的本质，是从数据中的真实box将相似大小的box归为一类，同时为了消除欧式距离对聚类的影响，利用IOU来进行聚类的评判指标。计算指标为

$$d(\text{box}, \text{centroids}) = 1 - \text{IOU}(\text{box}, \text{centroids})$$

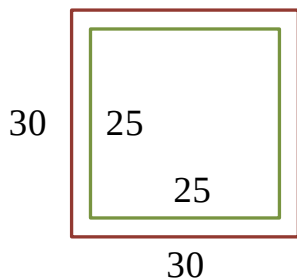


注意：计算IOU是将不同box的中心点重合，计算不同box之间的交并比

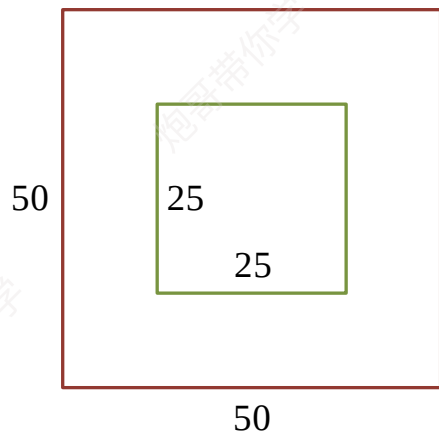
Yolov2改进点- 聚类利用欧式距离问题及其改进



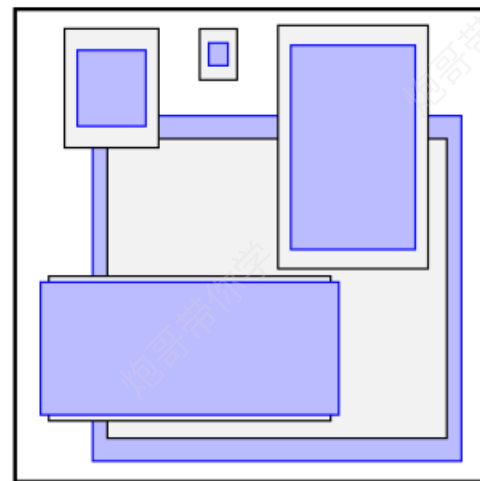
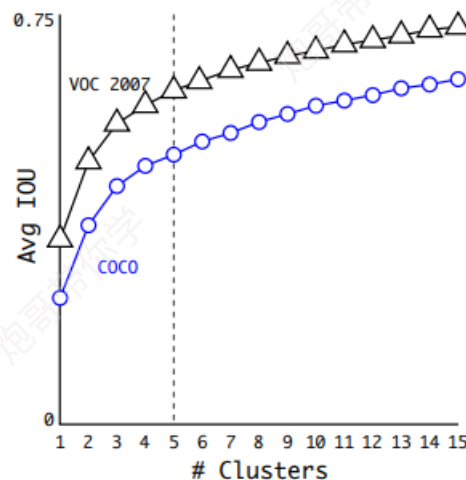
$$d(\text{box}, \text{centroids}) = 1 - (45 * 45) / (50 * 50) = 0.19$$



$$d(\text{box}, \text{centroids}) = 1 - (25 * 25) / (30 * 30) = 0.305$$



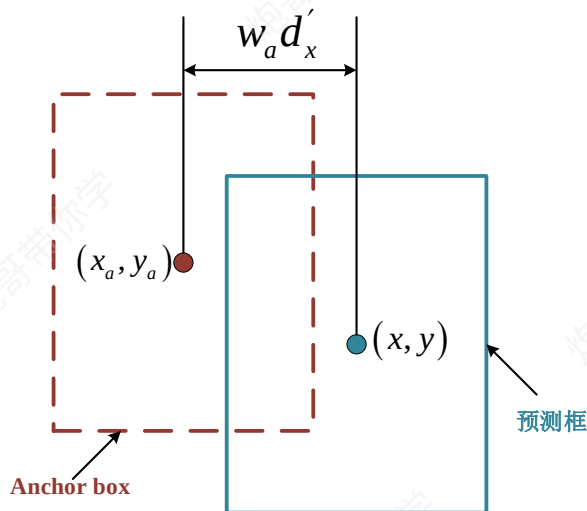
$$d(\text{box}, \text{centroids}) = 1 - (25 * 25) / (50 * 50) = 0.75$$



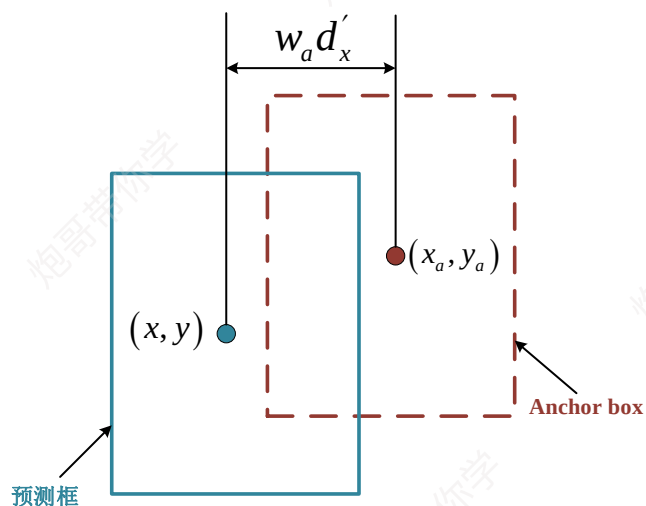
平衡复杂度和IOU之后，最终得到k值为5。可以从右边的聚类结果上看到5个聚类中心的宽高与手动精选的boxes是完全不同的，扁长的框较少，瘦高的框较多(黑丝框对应VOC2007数据集，紫色框对应COCO数据集)。这样就能明白为什么上面网络框架中的输出为什么是13x13x125了，因为通过聚类选用了5个anchor。

虽然说k值越大分的越细，差距也就越小，但是也不能设置太多的anchor boxes，而且在5之后曲线上升的就不是那么快了，所以选了一个折中的值5。

Yolov2改进点- location prediction:



情况1



情况2

YOLOv2借鉴RPN网络使用anchor boxes来预测边界框相对先验框的offsets。边界框的实际中心位置 (x, y) , 需要根据预测的坐标偏移值 (d'_x, d'_y) , 先验框的尺寸 (w_a, h_a) 来计算:

$$x = x_a + (d'_x \times w_a)$$

$$y = y_a + (d'_y \times h_a)$$

但是上面的公式是无约束的，预测的边界框很容易向任何方向偏移，因此每个位置预测的边界框可以落在图片任何位置，这导致模型的不稳定性，在训练时需要很长时间来预测出正确的offsets。

例如：

$d'_x = 1$ 则将box在x轴向右移动 w_a 。 $d'_x = -1$ 则将box在x轴向左移动 w_a 。y坐标如上同理，这就会导致预测框可能出现在图片的任何位置。

Yolov2改进点– location prediction:

YOLOv2弃用了这种RPN网络预测方式，而是沿用YOLOv1的方法，就是预测边界框中心点相对于对应cell左上角位置的相对偏移值，为了将边界框中心点约束在当前cell中，使用sigmoid函数处理偏移值，这样预测的偏移值在(0,1)范围内（每个cell的尺度看做1）。

$$b_x = \sigma(t_x) + c_x$$

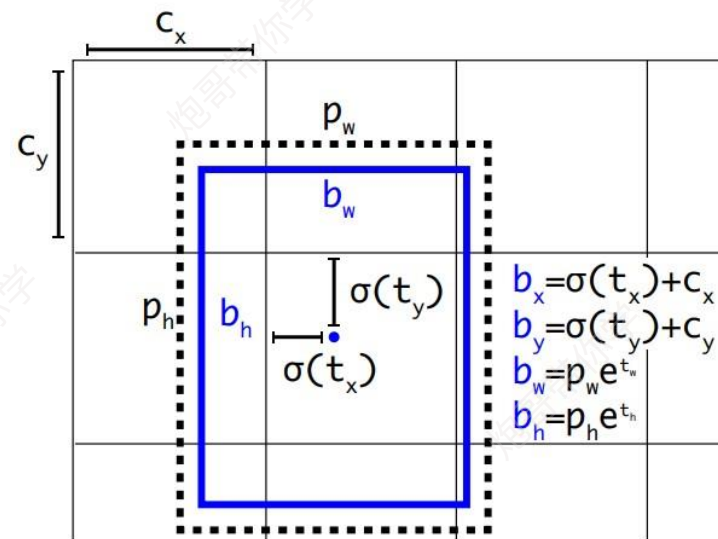
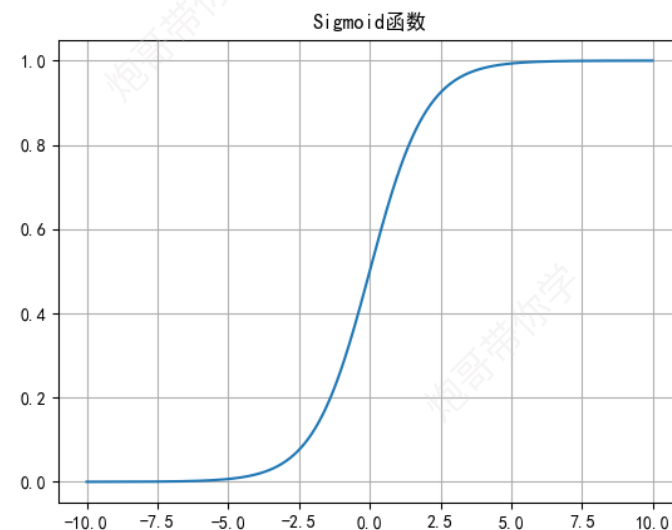
$$b_y = \sigma(t_y) + c_y$$

$$b_w = p_w e^{t_w}$$

$$b_h = p_h e^{t_h}$$

其中 (c_x, c_y) 为cell的左上角坐标，如图所示，在计算时每个cell的尺度为1。由于sigmoid函数的处理，边界框的中心位置会约束在当前cell内部，防止偏移过多。由于sigmoid函数的处理，边界框的中心位置会约束在当前cell内部，防止偏移过多。其中 e^{t_w} 和 e^{t_h} 没有过多约束，因为物体的大小是不受限制的，所以框的大小也没有约束。而 p_w 和 p_h 是先验框的宽度与长度，前面说过它们的值也是相对于特征图大小的，在特征图中每个cell的长和宽均为1。

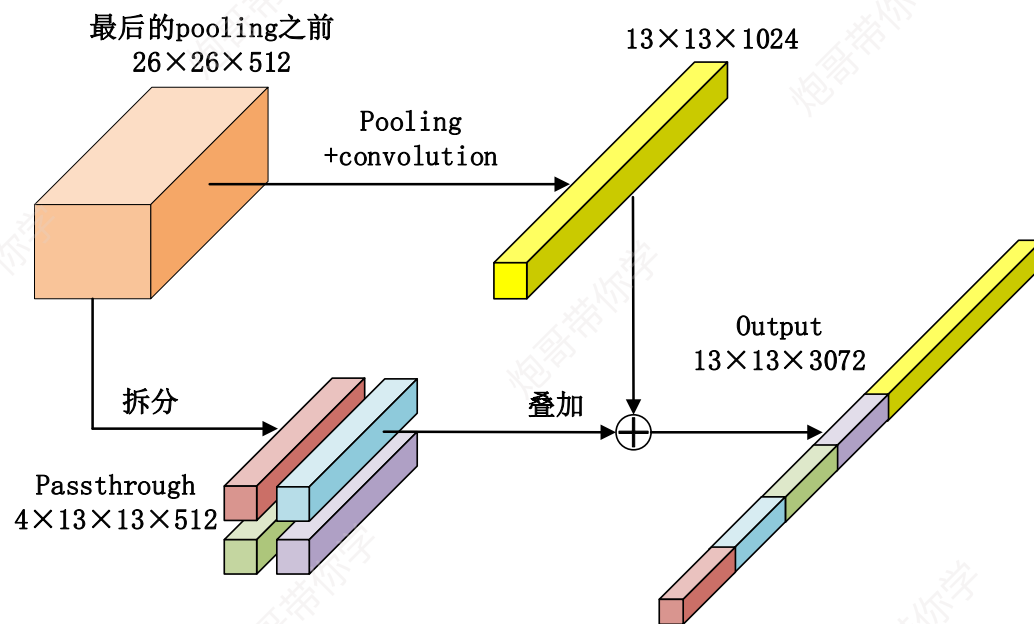
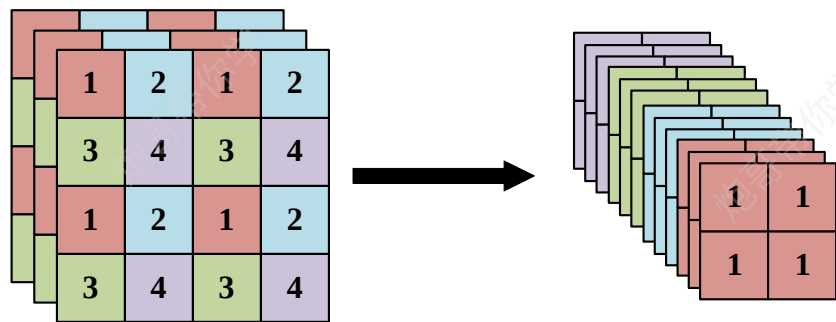
利用聚类 and 位置预测方法直接使MAP提升了4.8，提升巨大



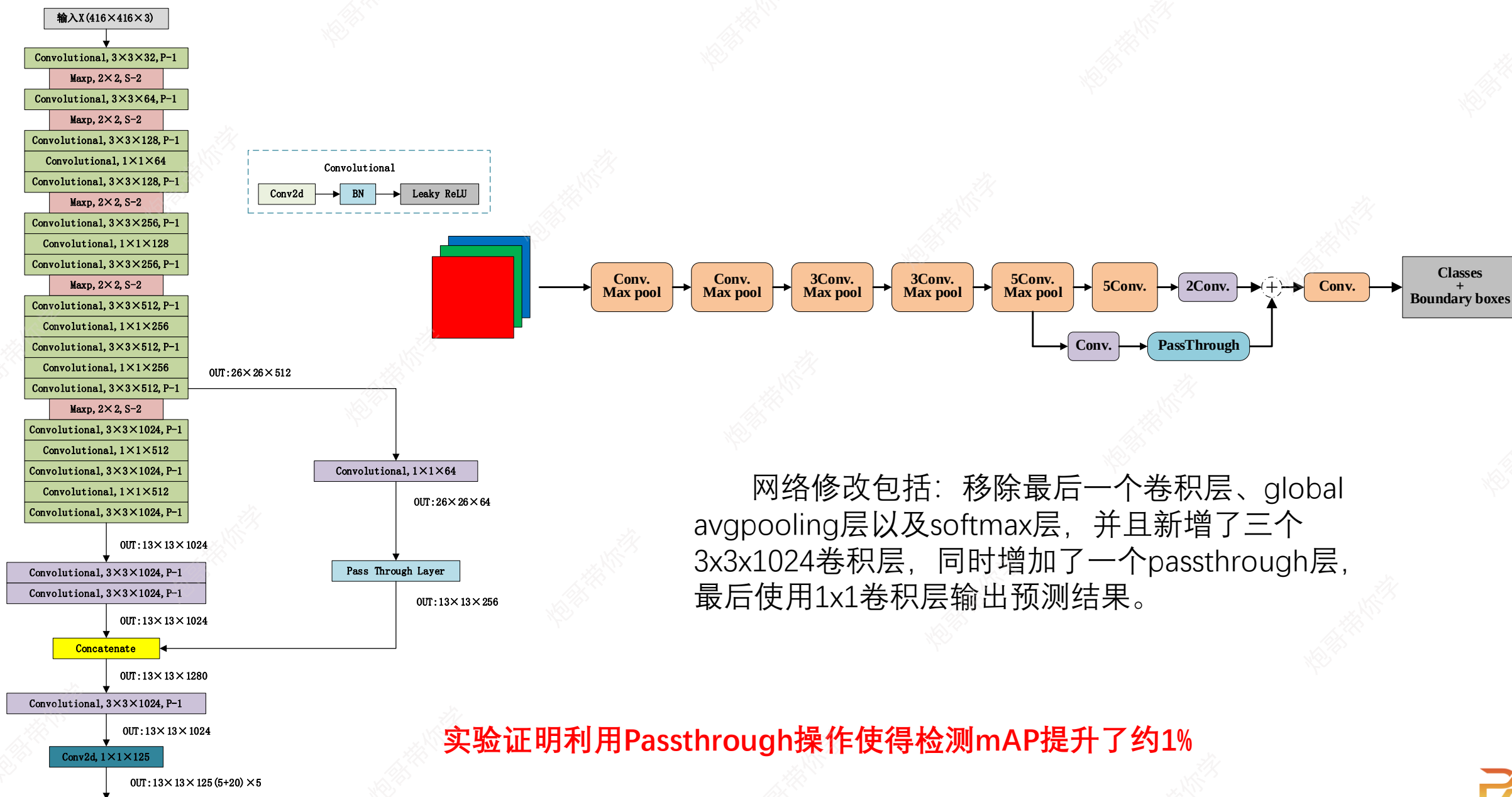
Yolov2改进点- Passthrough

Passthrough解决的问题：

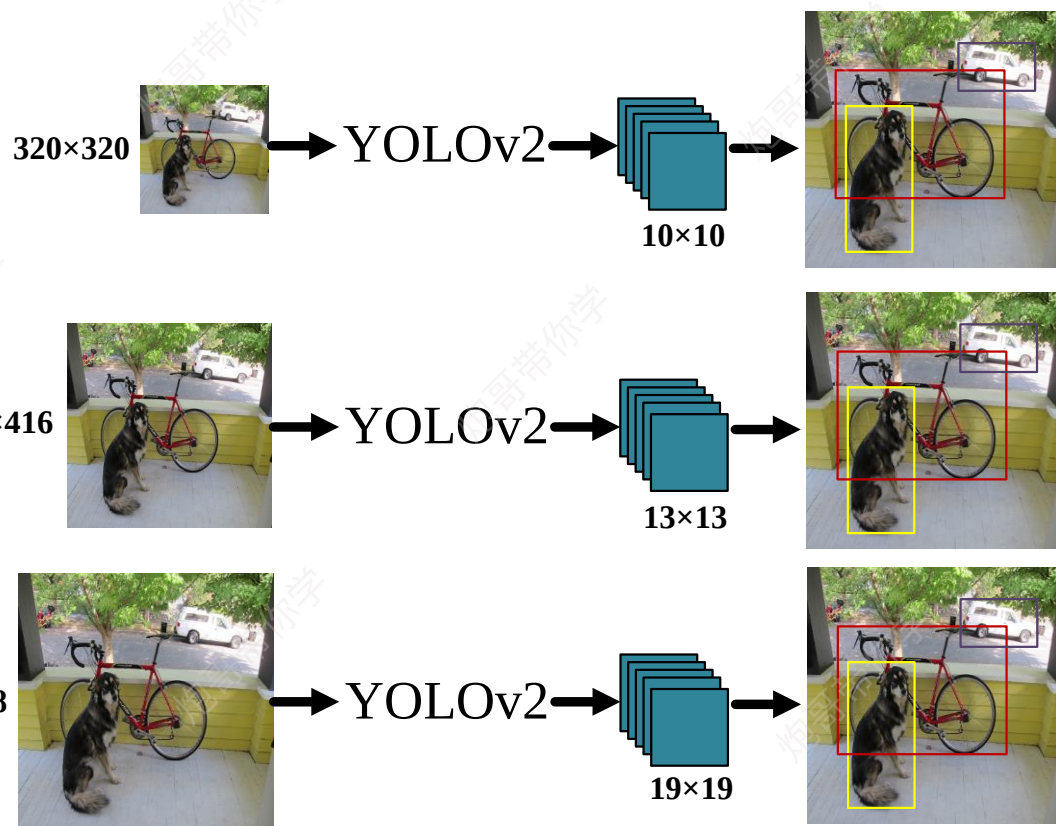
输入图像经过多层网络提取特征，最后输出的特征图中（比如YOLO2中输入416*416经过卷积网络下采样最后输出是13*13），较小的对象可能特征已经不明显甚至被忽略掉了。为了更好的检测出一些比较小的对象，最后输出的特征图需要保留一些更细节的信息。



Yolov2改进点- Passthrough (完整版Darknet-19-检测模型)

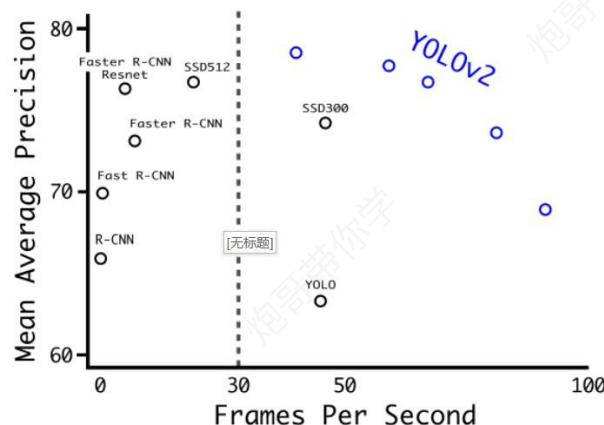


Yolov2改进点- multi-scale-training



作者引入了Multi-Scale Training, 简单讲就是在训练时输入图像的size是动态变化的, 注意这一步是在检测数据集上fine tune时候采用的。希望YOLOv2能够鲁棒地运行在不同尺寸的图像上, 所以我们将多尺度训练应用到模型中。

具体来讲, 在训练网络时, 每训练10个batch (可能是笔误), 网络就会随机选择另一种size的输入。作者采用从{320,352,...,608}的输入尺寸。



Detection Frameworks	Train	mAP	FPS
Fast R-CNN [5]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[15]	2007+2012	73.2	7
Faster R-CNN ResNet[6]	2007+2012	76.4	5
YOLO [14]	2007+2012	63.4	45
SSD300 [11]	2007+2012	74.3	46
SSD500 [11]	2007+2012	76.8	19
YOLOv2 288 × 288	2007+2012	69.0	91
YOLOv2 352 × 352	2007+2012	73.7	81
YOLOv2 416 × 416	2007+2012	76.8	67
YOLOv2 480 × 480	2007+2012	77.8	59
YOLOv2 544 × 544	2007+2012	78.6	40

实验证明多尺度训练使得检测mAP提升了约1.4

Yolov2改进点- hi-res detector

YOLOV2训练分为三个阶段:

第一阶段:

在ImageNet分类数据集上预训练Darknet-19, 此时模型输入为224x224, 共训练160个epochs。

第二阶段:

将网络的输入调整为448x448, 继续在ImageNet数据集上finetune分类模型, 训练10个epochs。

第三阶段:

1、修改Darknet-19分类模型为检测模型, 并在检测数据集上继续finetune网络(160轮)。网络修改包括: 移除最后一个卷积层、global avgpooling层以及softmax层, 并且新增了三个3x3x1024卷积层, 同时增加了一个passthrough层, 最后使用1x1卷积层输出预测结果。

2、输出的channels数为: $\text{num_anchors} \times (5 + \text{num_classes})$, 和训练采用的数据集有关系。由于anchors数为5, 对于VOC数据集输出的channels数就是125, 而对于COCO数据集则为425。以VOC为例, 同时采用多尺度数据训练, 采用从{320,352,...,608}的输入尺寸每10个batch随机选取一个尺寸送入网络中, 模型的输出为大小为{10,11,...,19}, 最终的预测矩阵为 ($\{10,11,...,19\}^2, 125$)

YOLOV2模型检测:

由于模型没有全连接层, 因此模型可以接受不同大小的输入图片, 当模型输入为416x416时, 测试集大小为76.8, 模型的输入为544x544, 测试集大小为78.6。

	YOLO									YOLOv2
batch norm?		✓	✓	✓	✓	✓	✓	✓	✓	✓
hi-res classifier?			✓	✓	✓	✓	✓	✓	✓	✓
convolutional?				✓	✓	✓	✓	✓	✓	✓
anchor boxes?				✓	✓					
new network?					✓	✓	✓	✓	✓	✓
dimension priors?						✓	✓	✓	✓	✓
location prediction?						✓	✓	✓	✓	✓
passthrough?							✓	✓	✓	✓
multi-scale?								✓	✓	✓
hi-res detector?									✓	✓
VOC2007 mAP	63.4	65.8	69.5	69.2	69.6	74.4	75.4	76.8		78.6

Detection Frameworks	Train	mAP	FPS
Fast R-CNN [5]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[15]	2007+2012	73.2	7
Faster R-CNN ResNet[6]	2007+2012	76.4	5
YOLO [14]	2007+2012	63.4	45
SSD300 [11]	2007+2012	74.3	46
SSD500 [11]	2007+2012	76.8	19
YOLOv2 288 × 288	2007+2012	69.0	91
YOLOv2 352 × 352	2007+2012	73.7	81
YOLOv2 416 × 416	2007+2012	76.8	67
YOLOv2 480 × 480	2007+2012	77.8	59
YOLOv2 544 × 544	2007+2012	78.6	40

高分辨率输入 (544x544) 的图片使得模型的mAP提升了约1.8

Yolov2损失函数

损失函数:

W、H、A为对应的cell 的对应的预选框，也
就是要遍历所有的预选框。

$$loss_t = \sum_{i=0}^W \sum_{j=0}^H \sum_{k=0}^A 1_{\text{Max IoU} < \text{Thresh}} \lambda_{\text{noobj}} * (-b_{ijk}^o)^2$$

计算背景box的置信度
误差

$$+ 1_{t < 12800} \lambda_{\text{prior}} * \sum_{r \in (x, y, w, h)} (\text{prior}_k^r - b_{ijk}^r)^2$$

预测框和先验框之间的位置
和大小误差，只在前12800
步之前计算

其中:

$$\lambda_{\text{noobj}} = 1$$

$$\lambda_{\text{obj}} = 5$$

$$\lambda_{\text{prior}} = 0.01$$

$$\lambda_{\text{coord}} = 1$$

$$\lambda_{\text{class}} = 1$$

$$+ 1_k^{\text{truth}} \left(\lambda_{\text{coord}} * \sum_{r \in (x, y, w, h)} (\text{truth}^r - b_{ijk}^r)^2 \right.$$

$$+ \lambda_{\text{obj}} * (IOU_{\text{truth}}^k - b_{ijk}^o)^2$$

$$+ \lambda_{\text{class}} * \left(\sum_{c=1}^c (\text{truth}^c - b_{ijk}^c)^2 \right) \Bigg]$$

预测框和真实框之间的位
置误差，大小误差，置信
度误差，类别误差

Yolov2损失函数

损失函数:

第一项loss是计算background的置信度误差。

$$1_{\text{Max IoU} < \text{Thresh}} \lambda_{\text{noobj}} * (-b_{ijk}^o)^2$$

其中, $1_{\text{Max IoU} < \text{Thresh}}$ 为判定条件, 计算各个**预测框**和所有**ground truth**的IOU值, 并且取最大值MaxIOU, 如果该值小于一定的阈值(YOLOv2使用的是0.6), 那么那么这个预测框就标记为background, 需要计算noobj的置信度误差。

如果该框预测的为背景, 我们则希望该框的置信度尽可能等于0, 因此, 标签为0, 模型输出的置信度为 b_{ijk}^o , 因此该项loss的计算为

$$(0 - b_{ijk}^o)^2 = (-b_{ijk}^o)^2$$

第二项是计算先验框与预测宽的坐标误差。

$$1_{t < 12800} \lambda_{\text{prior}} * \sum_{r \in (x, y, w, h)} (\text{prior}_k^r - b_{ijk}^r)^2$$

只在前12800个iterations间计算, 这项是在训练前期使预测框快速学习到先验框的形状; 使得模型在训练前期, 模型输出的预测框不至于太过于离谱, 毕竟先验框是从训练集中距离得来的, 本来就反应数据中物体的大小形状。

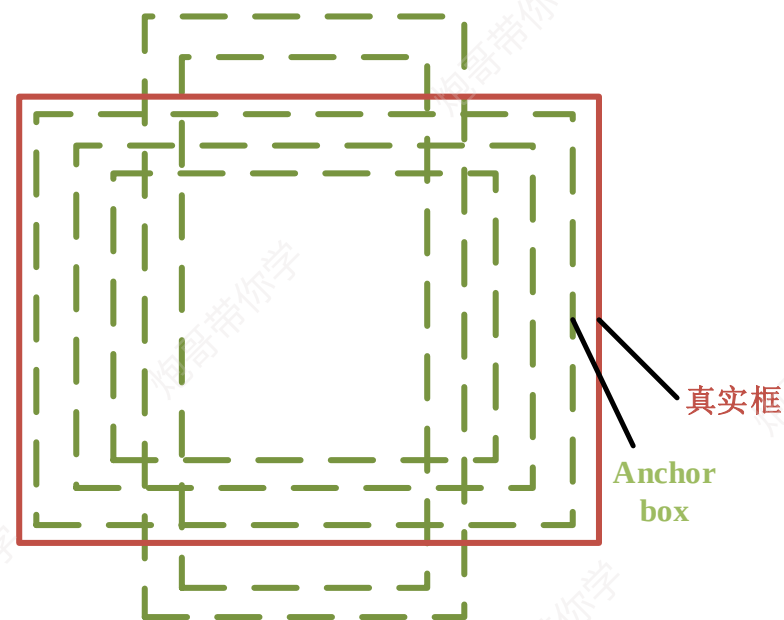
Yolov2损失函数

损失函数：

第三项计算的是有物体的预测框各部分的损失总和，包括坐标损失，置信度损失以及分类损失。

$$\begin{aligned} &+1_k^{\text{truth}} \left(\lambda_{\text{coord}} * \sum_{r \in (x,y,w,h)} \left(\text{truth}^r - b_{ijk}^r \right)^2 \right) \quad \text{坐标损失} \\ &\quad + \lambda_{\text{obj}} * \left(\text{IOU}_{\text{truth}}^k - b_{ijk}^o \right)^2 \quad \text{置信度损失} \\ &\quad + \lambda_{\text{class}} * \left(\sum_{c=1}^c \left(\text{truth}^c - b_{ijk}^c \right)^2 \right) \quad \text{类别损失} \end{aligned}$$

这里的匹配原则是指对于某个特定的真实框（标注框），首先要计算其中心点落在哪个cell上，然后计算这个 cell 的 5 个先验框和标注框的 IOU 值，计算 IOU 值的时候不考虑坐标只考虑形状，所以先将 Anchor boxes 和标注框的中心都偏移到同一位置，然后计算出对应的 IOU 值，IOU 值最大的先验框和标注框匹配，对应的预测框用来预测这个标注框。而对于没有标注框匹配的先验框，除去那些 Max_IOU 低于阈值的，其它就全部忽略。



Yolov2总结

优势:

关于yolov1中定位不准确和检测召回率较低的问题在yolov2中有了较好的解决。Yolov2中提出的如下几点在后续版本的yolo中有了较好的沿用。

- 1、anchor box（预选框）
- 2、Darknet网络架构

缺点:

- 1.小目标检测效果较差
- 2.整体检测效果还有待提高

	YOLO									YOLOv2
batch norm?		✓	✓	✓	✓	✓	✓	✓	✓	✓
hi-res classifier?			✓	✓	✓	✓	✓	✓	✓	✓
convolutional?				✓	✓	✓	✓	✓	✓	✓
anchor boxes?				✓	✓	✓	✓	✓	✓	✓
new network?					✓	✓	✓	✓	✓	✓
dimension priors?						✓	✓	✓	✓	✓
location prediction?						✓	✓	✓	✓	✓
passthrough?							✓	✓	✓	✓
multi-scale?								✓	✓	✓
hi-res detector?									✓	✓
VOC2007 mAP	63.4	65.8	69.5	69.2	69.6	74.4	75.4	76.8		78.6

